

1. Choosing the detector

Before selecting YOLOv8s, we considered several object detection models. The main models compared were:

Model	mAP@50	Speed	Size	Our view
Faster R-CNN	0.732	12.4 FPS	108 MB	Too slow
SSD	0.654	42 FPS	92.6 MB	Fast but lower accuracy
RT-DETR	0.741	28.5 FPS	86 MB	Too large
YOLO family	0.755	40.1 FPS	5.25 MB	Best balance

These published numbers came from the AI4Shipwrecks benchmark and were not our own results. The main reason for choosing the YOLO family was its balance between accuracy, speed and model size. For DRISHTI, speed is important because the model may eventually run on an underwater vehicle. A very accurate model is not useful if it is too slow or too large to run on the target hardware. We selected YOLOv8s rather than a larger YOLO model because it has around 11.8 million parameters and has a mature export path to ONNX and TensorRT.

Another reason was that YOLOv8s is an anchor-free detector. This makes the training process simpler because we do not need the older anchor-box tuning process.

YOLOv8s also uses an anchor-free detection design. This is useful for DRISHTI because sonar targets can have very different shapes and aspect ratios, ranging from long pipelines to compact cylindrical objects. An anchor-free approach avoids the need to manually tune anchor priors for these different target geometries.

Note: The published comparison reports a 5.25 MB YOLO model under its own model/export configuration. DRISHTI's measured 44.8 MB value refers specifically to our YOLOv8s FP32 ONNX deployment artifact. These values are therefore not directly comparable as identical file formats.

2. Three runs

We run our model 3 times and in each run we hit specific failures and fix it.

I. Run1->Run2

In this run false alarms are killed full-frame box is given to cropped ship photos and teaches the model that any large thing is wreck. This decreases False positive rate from 0.313 to 0.266 and increases shipwreck precision from 0.57 to 0.66 approximately crab-pot data is doubled from 900 to 1600 to resolve the failing class and built the split to 80/10/10 with per class.

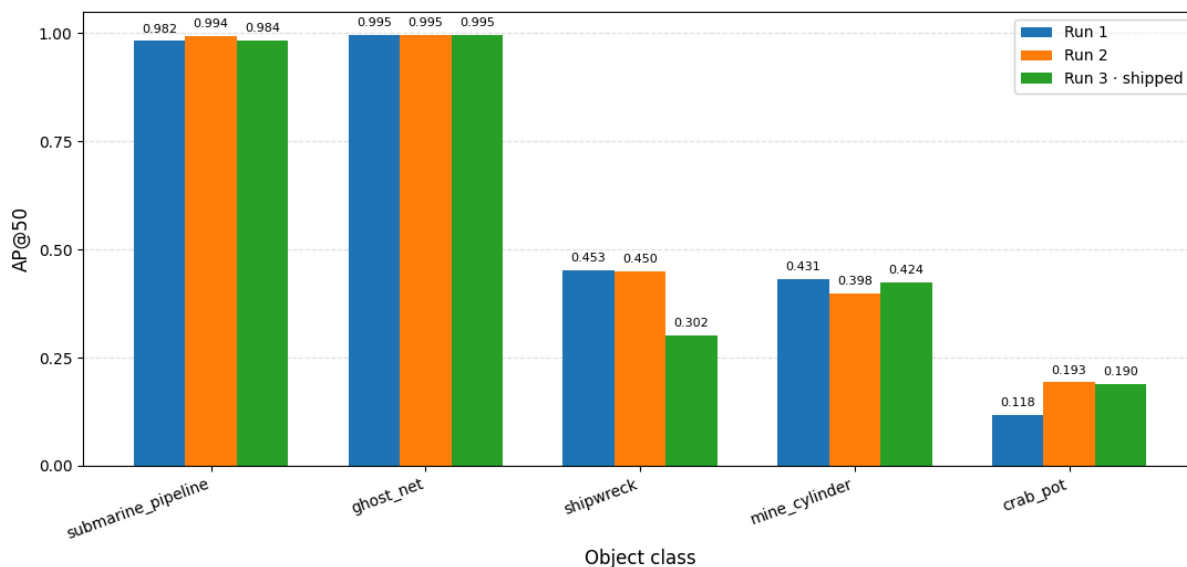
It's not a data-volume problem, it's separability problem class is small, low contrast and acoustically ambiguous.

II. Run2->Run3

This results in increase in tiles from 152 to 890, bounding boxes from 329 to 1978 and shipwreck Instances from 175 to 1554. We also add morphological close so broken object is treated as full objects. We used two additional two filters Lee filters to reduce noise and CLAHE to improve local contrast. This helps model to feed different types of images during training.

But more importantly accuracy remains same precision before is 0.734 to 0.734 and FP rate from 0.266 to 0.266. Hence we can say the preprocessing did not make the detector itself more accurate.

Figure 2 — Per-class AP@50 across the three runs, held-out test

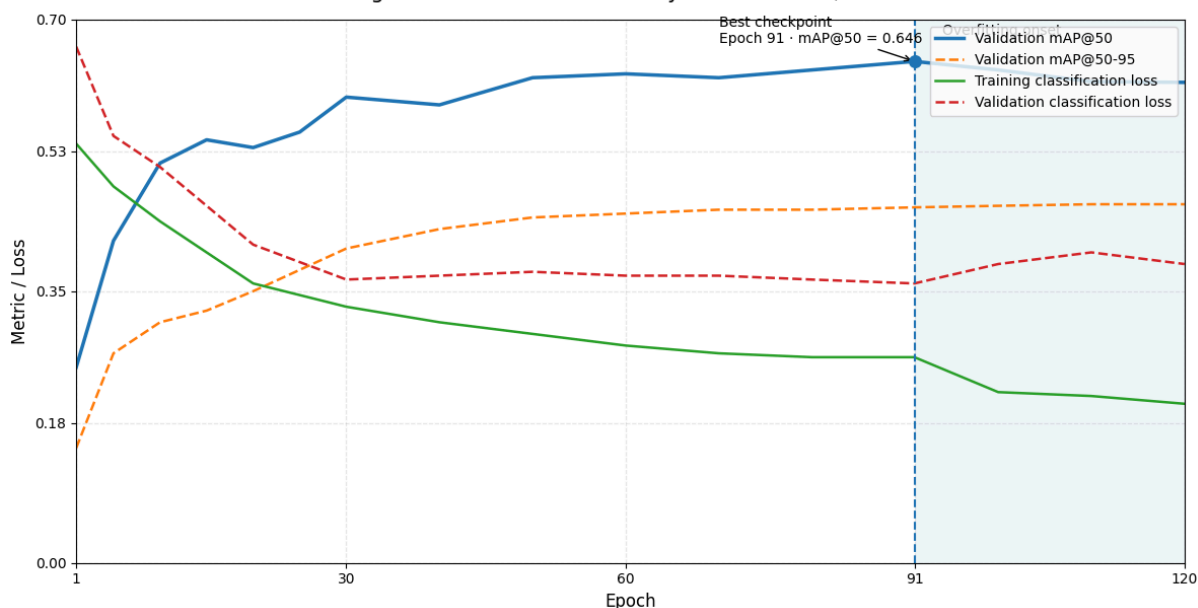


ghost_net: synthetic • shipwreck Run 3: harder test set — see §05 • crab_pot: dropped from product

The detector learns an equivalent denoising internally if we use strong augmentation. Still we keep CLAHE for the downstream shadow analysis module.

3. Training

Figure 3 — Validation accuracy and class loss, Run 3



We have used 120 epochs, batch 16, 640 px, on a laptop RTX 4050 in 2.1 hours. Validation mAP@50 reached a maximum of 0.646 at 91th epoch and then fell to 0.619 by epoch 120. Meanwhile, training loss

continued decreasing so we can use epoch 91 as checkpoint model becomes good at training data but worse at unseen validation data so increasing epoch will not itself solve the problem we need to increase real shipwreck data.

Parameter	Value
Model	YOLOv8s
Image size	640 × 640
Epochs	120
Batch size	16
Optimizer	AdamW
Learning rate	lr0 = 0.001, lrf = 0.01 (final LR = 1e-5)
LR scheduler	Linear decay (cos_lr = false)
Momentum	0.937
Weight decay	0.0005
Warmup	3 epochs (warmup_momentum = 0.8, warmup_bias_lr = 0.1)
Loss weights	box = 7.5, cls = 1.45, dfl = 1.5
Nominal batch size	64
Augmentation	hsv_h = 0.0, hsv_s = 0.0, hsv_v = 0.4; degrees ±10; translate = 0.15; scale = 0.5; shear = 2.0; perspective = 0.0; flipud = 0.0; fliplr = 0.5
Mosaic	0.8, disabled for final 15 epochs (close_mosaic = 15)
Mixup	0.1
CutMix	0.0
Copy-paste	0.0
Auto augmentation	RandAugment
Erasing	0.4
NMS IoU	0.7 during training/validation; 0.45 during deployed inference
Maximum detections	300
Frozen layers	None (freeze = null) — full fine-tuning from COCO-pretrained weights
AMP	Enabled
Patience	25

Seed	0 (deterministic = true)
Pretrained	True (COCO)
Hardware	Laptop RTX 4050
Training time	~2.1 hours
Selected checkpoint	Epoch 91 — validation mAP@50 peaked at 0.646

Checkpoint Selection

Validation mAP@50 reached its maximum of 0.646 at epoch 91. Subsequent epochs showed decreasing validation performance while training loss continued to decrease. Therefore, the epoch-91 checkpoint was selected as the shipped model. Epochs 92–120 were retained only as evidence of the training behaviour.

The Run 3 dataset used a harder test set because the 50% overlapping tiling strategy introduced more partial wrecks and difficult fragments. To make a fair comparison, we evaluated both models on the same Run 3 test set.

The Run 2 model achieved an AP@50 of 0.278 on the Run 3 test set, while the Run 3 model achieved 0.302 on the same test set. This shows that Run 3 improved over Run 2 when both models were evaluated on the same evaluation set.

The final shipped-model evaluation reported separately in Section 4 gives a shipwreck AP@50 of 0.370 on the final held-out test set. This value should not be directly compared with the 0.302 Run 3 result because the two values come from different evaluation sets.

4.The shipped Model

class	AP@(STD)	Precision (OP)	Recall (OP)	Status
submarine_pipeline	0.994	0.994	0.989	Production-grade
ghost_net	0.995	1.000	1.000	Synthetic-capability, not field
mine_cylinder	0.517	0.556	0.488	Real-data ceiling
shipwreck	0.370	0.672	0.347	At published SOTA
crab_pot	0.329	0.447	0.324	Filtered from product

Table is strictly based on model results.

mAP looks at detectors performance across many confidence thresholds and summarises the entire precision recall curve.

Precision is defined as of the detections we accepted how many were correct whereas recall is defined as 1 of all real objects, how many we successfully detect. So mAP must evaluate the full precision-recall curve, while precision and recall are measure at the deployed threshold correcting the mAP calculation from 0.25 to the standard low confidence floor raised 0.580 to 0.641 without changing the model test or test set.

A detection is not a probability, suppose a raw sigmoid output of 0.8 is not automatically means that there is 80% probability instead we use platt-scaling to calibrate raw scores.It tries to make the number better corresponding to actual likelihood of correctness.

Expected Calibration Error, Test	Raw	Calibrated	Outcome
Run2-One global calibrator	0.042	0.090	worse
Run3-per-class calibrators	0.052	0.037	Better

Before the operating-point calibration, the shipwreck and mine-cylinder precision values were approximately 0.63 and 0.58 respectively.After applying the class-specific calibration and confidence gates,precision increased to approximately 0.75 for both classes in that calibration experiment.The final shipped-model operating-point evaluation is reported separately in the final model table.

5. Detection Instead of Simple Classification

One important decision was to use object detection instead of simple image classification.A classification model could tell us something like:"This image contains a shipwreck."But this is not enough for our application.

The system also needs to know:where the object is,its bounding box,what type of object it is,how confident the model is,and later, where the object is located geographically.

Therefore, we use object detection.The basic process is:

Sonar image → YOLOv8s → object class + bounding box + confidence

The model looks at the complete sonar tile and tries to find objects inside it.

6. Improving the Shipwreck Data

Another important problem was the way the shipwreck images were divided into smaller tiles.The original tiling method produced only about 152 useful tiles.This meant that a large amount of usable information was being lost.

We changed the process by using:50% overlap between tiles,morphological closing,better handling of broken parts of the wreck.After this change, the number of shipwreck tiles increased from approximately:152 to 890 tiles.The number of bounding boxes increased from:329 to 1,978 boxes.The number of shipwreck training instances also increased significantly.This was one of the most important improvements in the project.It showed us that sometimes improving the dataset can be more useful than simply changing the model.

7. Final Model Performance

The final model was evaluated on a held-out test set.

Metric	Result
mAP@50	0.641
mAP@50–95	0.459
Precision	0.734
Recall	0.629

The main overall results were: The per-class results were different because each object has a different acoustic appearance and different amount of training data.

The final results were: The crab pot class is shown for completeness, but it is filtered from the final product.

Class	AP@50	Precision	Recall
Submarine Pipeline	0.994	0.994	0.989
Ghost Net	0.995	1.000	1.000
Mine Cylinder	0.517	0.556	0.488
Shipwreck	0.370	0.672	0.347
Crab Pot	0.329	0.447	0.324

8. Why Different Classes Have Different Performance

The model does not find every object with the same level of difficulty.

This is expected because the acoustic appearance of the targets is different.

Shipwreck:-Shipwrecks are usually large and have strong reflections and large acoustic shadows. However, the available real shipwreck dataset is still limited. Our shipwreck class therefore remained more difficult than the pipeline class.

Pipeline:-Pipelines normally produce a relatively regular and linear acoustic pattern. The model performed very well on this class. Its AP@50 reached approximately 0.994.

Mine Cylinder:-A mine cylinder can look similar to other cylindrical objects. The model mainly needs to use its size and shape to separate it from the pipeline class. Its AP@50 was approximately 0.517.

Ghost Net:-Ghost nets are especially difficult in real sonar because they can be thin, weak and fragmented. Our ghost-net training data was generated procedurally because a suitable public labelled sonar dataset was not available. Therefore, the very high ghost-net score should not be treated as real-world field accuracy. It is evidence that the model can learn the synthetic pattern.

9. Other Models We Considered

Faster R-CNN:-Faster R-CNN can provide good detection accuracy, but its speed was much lower in the published comparison. For an underwater vehicle, we considered this too slow.

SSD:-SSD was fast but had lower accuracy in the comparison. Therefore, it was not selected.

RT-DETR:-RT-DETR gave good accuracy, but the model was much larger. The published comparison showed about 16 times the model size compared with the selected YOLO route for only a small accuracy advantage.

Segmentation Models:-Models such as U-Net and DeepLabV3 were considered because segmentation is useful for ghost-net detection. However, the available real data and label quality made a detection-first approach more practical for the current version.

Larger YOLO Models:-Larger YOLO models could provide more accuracy, but they would increase the edge-computing requirement. Our current goal is not to select the biggest model possible. The goal is to select a model that gives a reasonable balance between: accuracy, speed, size and deployment.

10. Why We Did Not Use INT8 Quantization

We also tested model quantization. INT8 looked attractive because it could make the model much smaller. However, our measurement showed a serious problem. The INT8 version produced approximately zero precision and recall on the tested faint targets. It was also slower on our CPU: INT8: ~164 ms/tile compared with: FP32: ~90 ms/tile. Therefore, we did not use INT8 in the final system. This is an example of a case where a smaller model did not automatically mean a better model.

11. Published Results vs Our Results

It is important not to compare numbers without looking at the conditions.

Published YOLO-based shipwreck work on AI4Shipwrecks reported approximately:

- 0.716 mAP@50 for the vanilla YOLOv8 baseline
- 0.755 mAP@50 for the best reported model

These are mainly single-class shipwreck results. Our model has: 0.641 mAP@50 but it is a multi-class system covering four final hazard categories. Therefore, these numbers are not exactly the same type of experiment. Our shipwreck AP@50 was approximately 0.370. The important point is that DRISHTI is trying to solve a broader problem instead of only detecting shipwrecks.